D. Manipuler les conteneurs

ATELIER PRATIQUE GUIDÉ

Conteneurisation d'une application microservices avec Docker

Conteneuriser une application Node.js.

Connecter l'application à une base de données MongoDB.

Mettre en place une communication asynchrone avec RabbitMQ.

Configurer un reverse proxy Nginx.

Orchestrer plusieurs services avec Docker Compose.

1. Structure du projet

Créer la structure suivante :

```
microservices-docker/
|
 ├── user-service/
|    ├── server.js
|    ├── package.json
|    └── Dockerfile
|
 ├── nginx/
|    └── nginx.conf
|
└── docker-compose.yml
```

2. Création du microservice Node.js

2.1 Fichier package.json

```
{
  "name": "user-service",
  "version": "1.0.0",
  "description": "Microservice de gestion des utilisateurs",
```

```
  "main": "server.js",
  "scripts": {
    "start": "node server.js"
  },
  "dependencies": {
    "express": "^4.18.2",
    "mongoose": "^7.0.0",
    "amqplib": "^0.10.3"
  }
}
```

Explication

- ***name*** *: Nom du projet.*
- ***main*** *: Fichier principal.*
- ***scripts.start*** *: Commande de démarrage.*
- ***express*** *: Framework web.*
- ***mongoose*** *: Interaction avec MongoDB.*
- ***amqplib*** *: Communication avec RabbitMQ.*

2.2 Fichier server.js

```
// Importation des modules nécessaires
const express = require('express');       // Framework pour créer le serveur web
const mongoose = require('mongoose');     // Bibliothèque pour interagir avec MongoDB
const amqp = require('amqplib');          // Client RabbitMQ pour la messagerie

// Création de l'application Express
const app = express();

// Middleware permettant de lire les données JSON envoyées dans les requêtes HTTP
app.use(express.json());

// ========================
// Connexion à MongoDB
```

```
// ======================

// Connexion à la base de données MongoDB.
// Le nom "mongo" correspond au nom du service défini dans docker-compose.
mongoose.connect('mongodb://mongo:27017/users_db', {
  useNewUrlParser: true,
  useUnifiedTopology: true
})
.then(() => console.log(' Connecté à MongoDB'))
.catch(err => console.error(' Erreur MongoDB :', err));

// ======================
// Définition du schéma utilisateur
// ======================

// Création d'un schéma Mongoose pour représenter un utilisateur
const userSchema = new mongoose.Schema({
  name: String,     // Nom de l'utilisateur
  email: String     // Email de l'utilisateur
});

// Création du modèle basé sur le schéma
const User = mongoose.model('User', userSchema);

// ======================
// Connexion à RabbitMQ
// ======================

let channel; // Canal de communication avec RabbitMQ

async function connectRabbitMQ() {
  try {
    // Connexion au serveur RabbitMQ (nom du service docker-compose)
    const connection = await amqp.connect('amqp://rabbitmq');

    // Création d'un canal de communication
    channel = await connection.createChannel();
```

```
// Création (ou vérification) de la file d'attente "user_created"
    await channel.assertQueue('user_created');

    console.log(' Connecté à RabbitMQ');
  } catch (error) {
    console.error(' Erreur RabbitMQ :', error);
  }
}

// Appel de la fonction de connexion
connectRabbitMQ();

// ========================
// Routes de l'API REST
// ========================

// Route GET : récupérer tous les utilisateurs
app.get('/users', async (req, res) => {
  const users = await User.find(); // Récupération des utilisateurs depuis MongoDB
  res.json(users);               // Envoi de la réponse au format JSON
});

// Route POST : ajouter un nouvel utilisateur
app.post('/users', async (req, res) => {
  const user = new User(req.body); // Création d'un nouvel utilisateur
  await user.save();             // Sauvegarde dans la base de données

  // Envoi d'un message à RabbitMQ pour notifier la création
  if (channel) {
    channel.sendToQueue(
      'user_created',
      Buffer.from(JSON.stringify(user))
    );
  }

  res.status(201).json(user); // Réponse avec le statut 201 (créé)
});
```

```
// Route racine pour tester le service
app.get('/', (req, res) => {
  res.send(' User Service opérationnel');
});

// Démarrage du serveur sur le port 3001
app.listen(3001, () => {
  console.log(' User Service démarré sur le port 3001');
});
```

3. Création du Dockerfile

Fichier user-service/Dockerfile

```
# Utilisation d'une image officielle Node.js comme base
FROM node:18

# Définition du répertoire de travail à l'intérieur du conteneur
WORKDIR /app

# Copie des fichiers package.json et package-lock.json
# Cette étape permet d'installer les dépendances avant de copier tout le code
COPY package*.json ./

# Installation des dépendances du projet
RUN npm install

# Copie de tous les fichiers du projet dans le conteneur
COPY . .

# Indication du port utilisé par l'application
EXPOSE 3001

# Commande exécutée au démarrage du conteneur
CMD ["npm", "start"]
```

Explication des instructions Dockerfile

Instruction	Rôle
FROM	Spécifie l'image de base
WORKDIR	Définit le dossier de travail
COPY	Copie les fichiers
RUN	Exécute une commande
EXPOSE	Indique le port utilisé
CMD	Commande de démarrage

4. Configuration du Reverse Proxy Nginx

Fichier nginx/nginx.conf

```
# Bloc principal des événements (obligatoire)
events {}

http {
  server {
    # Nginx écoute sur le port 80
    listen 80;

    # Route pour accéder au microservice utilisateur
    location /users/ {
      # Redirection des requêtes vers le service Node.js
      proxy_pass http://user-service:3001/;

      # Transmission des informations du client au service backend
      proxy_set_header Host $host;
      proxy_set_header X-Real-IP $remote_addr;
    }

    # Route racine pour tester Nginx
    location / {
      return 200 'Nginx Reverse Proxy opérationnel';
        add_header Content-Type text/plain;
```

```
      }
    }
  }
```

5. Orchestration avec Docker Compose

Fichier docker-compose.yml

```
version: '3.9'

services:
  # ========================
  # Service MongoDB
  # ========================
  mongo:
    image: mongo:6              # Image officielle MongoDB
    container_name: mongo
    ports:
      - "27017:27017"           # Exposition du port MongoDB
    volumes:
      - mongo_data:/data/db      # Persistance des données

  # ========================
  # Service RabbitMQ
  # ========================
  rabbitmq:
    image: rabbitmq:3-management  # Image avec interface web
    container_name: rabbitmq
    ports:
      - "5672:5672"             # Port pour la communication AMQP
      - "15672:15672"           # Interface d'administration web

  # ========================
  # Microservice Node.js
  # ========================
  user-service:
    build: ./user-service        # Construction de l'image à partir du Dockerfile
```

```
  container_name: user-service
  ports:
    - "3001:3001"            # Exposition du port du service
  depends_on:
    - mongo                  # Démarre après MongoDB
    - rabbitmq               # Démarre après RabbitMQ

# ========================
# Reverse Proxy Nginx
# ========================
nginx:
  image: nginx:latest
  container_name: nginx
  ports:
    - "80:80"                # Point d'entrée de l'application
  volumes:
    - ./nginx/nginx.conf:/etc/nginx/nginx.conf
  depends_on:
    - user-service           # Nginx démarre après le microservice

# ========================
# Volume pour la persistance
# ========================
volumes:
  mongo_data:
```

6. Démarrage de l'application

Construire et démarrer les conteneurs

```
docker compose up -d --build
```

Vérifier les conteneurs en cours

```
docker compose ps
```

Consulter les journaux

```
docker compose logs -f
```

Arrêter l'application

docker compose down

7. Tests de l'application

Élément	***URL***
Nginx	*http://localhost*
API utilisateurs	*http://localhost/users*
RabbitMQ (interface)	*http://localhost:15672*

Test avec Postman

Ajouter un utilisateur

- ***Méthode :*** *POST*
- ***URL :*** *http://localhost/users*
- ***Body (JSON) :***

```
{
  "name": "Ahmed",
  "email": "ahmed@mail.com"
}
```

Récupérer les utilisateurs

- ***Méthode :*** *GET*
- ***URL :*** *http://localhost/users*

8. Exercices

Exercice 1

Ajouter un champ age au schéma utilisateur.

Exercice 2

Créer une nouvelle file RabbitMQ user_deleted et envoyer un message lors de la suppression d'un utilisateur.

Exercice 3

Ajouter un second microservice (ex : product-service) et le configurer dans Nginx et Docker Compose.

Exercice 4

Modifier la configuration Nginx pour ajouter une route /api/users.